

BIG DATA ENGINEERING — HANDS-ON LAB

# Kafka Practice Lab

From Hadoop to Kafka: brokers, topics, producers, consumers,  
and consumer groups — with extra command examples

Hadoop

ZooKeeper

Broker

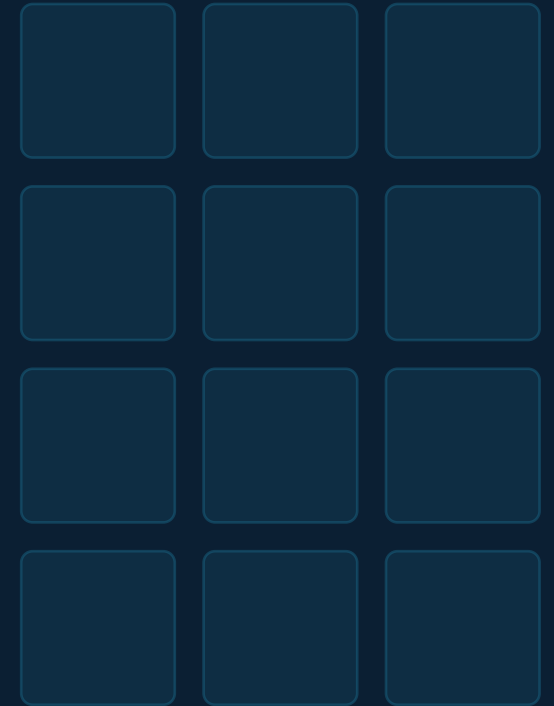
Topics

Producer

Consumer

Groups

*Ubuntu · Apache Kafka + ZooKeeper · Hadoop-based environment*



# The full pipeline we'll practice

One command at a time — verify each layer before moving to the next.

**1**  
Hadoop (HDFS + YARN)

**2**  
ZooKeeper

**3**  
Kafka Broker

**4**  
Create Topic

**5**  
List Topics

**6**  
Describe Topic

**7**  
Producer → Send

**8**  
Consumer → Read

**9**  
Consumer Groups

**10**  
Delete Topic

**11**  
Stop Kafka

**12**  
Stop ZooKeeper

**1**

# Check the Hadoop environment

Confirm Java, Hadoop, and running daemons before touching Kafka.

```
● ● ● hadoop-check.sh

# 1. Confirm Java is installed
$ java -version

# 2. Confirm Hadoop is installed
$ hadoop version

# 3. Confirm Hadoop daemons are up
$ jps
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
```

## If nothing is running

```
● ● ● start-hadoop.sh

$ start-dfs.sh
$ start-yarn.sh
$ jps
```

## Extra example — check ports in use:

```
sudo netstat -tulpn | grep -E "9000|8088"
```

2

## Start ZooKeeper (Terminal 1)

Kafka brokers register themselves in ZooKeeper — it must be up first.

```
terminal-1

$ cd ~/kafka
$ ls
bin  config  libs  logs

# Start the ZooKeeper server — keep this terminal open
$ bin/zookeeper-server-start.sh config/zookeeper.properties
...binding to port 0.0.0.0/0.0.0.0:2181
```

### Extra examples

#### Change the client port

Edit `clientPort=2181` in  
`config/zookeeper.properties`

#### Run ZooKeeper in the background

```
bin/zookeeper-server-start.sh -daemon
config/zookeeper.properties
```

#### View ZooKeeper logs

```
tail -f logs/zookeeper.out
```

## 3 Verify ZooKeeper (Terminal 2)

```
terminal-2
```

```
$ jps
```

```
QuorumPeerMain
```

### What QuorumPeerMain confirms

ZooKeeper is up and coordinating cluster metadata: broker registration, controller election, and topic/partition configuration — all of which Kafka reads from ZooKeeper before it can accept connections.

### Extra example — inspect ZooKeeper directly

```
zookeeper-shell
```

```
$ bin/zookeeper-shell.sh localhost:2181
```

```
# List registered broker IDs
```

```
ls /brokers/ids
```

```
[0]
```

```
# See broker connection details
```

```
get /brokers/ids/0
```

```
{"listener_security_protocol_map"...
```

```
# List all topics known to ZK
```

```
ls /brokers/topics
```

```
quit
```

## 4 Start & verify the Kafka broker

```
terminal-2

# Terminal 2 - start the broker, keep it open
$ cd ~/kafka
$ bin/kafka-server-start.sh config/server.properties
...[KafkaServer id=0] started
```

```
terminal-3

# Terminal 3 - confirm broker + ZooKeeper + Hadoop
$ jps
Kafka
QuorumPeerMain
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
```

**Hadoop — HDFS + YARN**



**ZooKeeper — coordination**



**Kafka Broker — :9092**

**Extra example — verify port is listening:**

```
check-broker

$ nc -zv localhost 9092
Connection to localhost 9092 port succeeded
```

## 6

# Create topics

```
● ● ● create-orders.sh

# Base exercise: topic "orders"
$ bin/kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --partitions 3 \
  --replication-factor 1
Created topic orders.
```

## orders — layout

Partition 0

Partition 1

Partition 2

## Extra examples

```
● ● ● more-topics.sh

# "payments" — more partitions
$ bin/kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic payments \
  --partitions 5 --replication-factor 1

# "app-logs" — with a retention config
$ bin/kafka-topics.sh --create \
  --bootstrap-server localhost:9092 \
  --topic app-logs --partitions 2 \
  --replication-factor 1 \
  --config retention.ms=3600000

# Add partitions later (can't decrease)
$ bin/kafka-topics.sh --alter \
  --bootstrap-server localhost:9092 \
  --topic orders --partitions 6
```

## 7

# List & describe topics

● ● ● list-topics.sh

```
$ bin/kafka-topics.sh --list \  
  --bootstrap-server localhost:9092  
orders
```

● ● ● describe-topic.sh

```
$ bin/kafka-topics.sh --describe \  
  --bootstrap-server localhost:9092 \  
  --topic orders  
Topic: orders    PartitionCount: 3  
ReplicationFactor: 1  
  Partition: 0   Leader: 0   Replicas: 0   Isr: 0  
  Partition: 1   Leader: 0   Replicas: 0   Isr: 0  
  Partition: 2   Leader: 0   Replicas: 0   Isr: 0
```

## Extra examples

● ● ● topic-inspect.sh

```
# Describe every topic at once  
$ bin/kafka-topics.sh --describe \  
  --bootstrap-server localhost:9092
```

```
# Only under-replicated partitions  
$ bin/kafka-topics.sh --describe \  
  --bootstrap-server localhost:9092 \  
  --under-replicated-partitions
```

```
# Read a topic's configs only  
$ bin/kafka-configs.sh --describe \  
  --bootstrap-server localhost:9092 \  
  --entity-type topics \  
  --entity-name orders
```

## 9

# Producer — send messages

```
producer.sh

$ bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders
>Order 1
>Order 2
>Order 3
>Order 4
# Ctrl+C to exit the producer
```

## Producer → Broker → Topic



## Extra examples

```
producer-extra.sh

# Send key:value pairs (key sets partition)
$ bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --property "parse.key=true" \
  --property "key.separator=:"
>1001:Order 1 - Cairo
>1002:Order 2 - Giza

# Produce a JSON message
>{"order_id":1003,"amount":49.90}

# Produce from a file instead of typing
$ bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders < orders.txt
```

## 10

# Consumer — read messages

```
● ● ● consumer.sh

$ bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning
Order 1
Order 2
Order 3
Order 4
# Ctrl+C to stop consuming
```

## --from-beginning

Reads every retained message in the topic, starting at the earliest offset. Without this flag, the consumer only sees messages produced after it connects.

## Extra examples

```
● ● ● consumer-extra.sh

# Show keys alongside values
$ bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders --from-beginning \
  --property print.key=true \
  --property key.separator=:
1001:Order 1 - Cairo

# Stop automatically after N messages
$ bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders --from-beginning \
  --max-messages 2
```

*Also useful: add --partition 0 to read a single partition only.*

# Consumer groups in action

A consumer group splits a topic's partitions across its members — no two members in the same group read the same partition.

```

● ● ● consumer-group.sh

# Join topic "orders" (3 partitions) as group "billing"
$ bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --group billing

# Run this again in a 2nd terminal — same group.
# Kafka rebalances: partitions split across both.
```

## Rebalanced assignment (2 consumers, 3 partitions)

**Consumer A**  
Partition 0  
Partition 1

**Consumer B**  
Partition 2

## Extra examples — kafka-consumer-groups.sh

```

● ● ● consumer-groups.sh

# List groups
$ bin/kafka-consumer-groups.sh --list \
  --bootstrap-server localhost:9092
billing

# Describe lag per partition
$ bin/kafka-consumer-groups.sh \
  --describe --group billing \
  --bootstrap-server localhost:9092
```

| TOPIC  | PARTITION | LAG | CONSUMER-ID |
|--------|-----------|-----|-------------|
| orders | 0         | 0   | consumer-A  |
| orders | 2         | 0   | consumer-B  |

Reset offsets: `--reset-offsets --to-earliest --execute` (add `--group/--topic`).

## 11

# Delete a topic

```
delete-topic.sh

# Confirm it exists first
$ bin/kafka-topics.sh --list \
  --bootstrap-server localhost:9092
orders

# Delete it
$ bin/kafka-topics.sh --delete \
  --bootstrap-server localhost:9092 \
  --topic orders
Topic orders is marked for deletion.

$ bin/kafka-topics.sh --list \
  --bootstrap-server localhost:9092
(ordere no longer listed)
```

## Gotcha

If deletion silently fails, `delete.topic.enable` may be set to `false` in `config/server.properties`. Set it to `true` and restart the broker for `--delete` to actually remove the topic instead of just marking it.

## Extra example — delete more than one topic

```
for t in payments app-logs; do
  bin/kafka-topics.sh --delete --bootstrap-server
localhost:9092 --topic $t
done
```

## BONUS

# More Kafka commands worth knowing

### Config

```
bin/kafka-configs.sh --alter --entity-type topics --entity-name orders --add-config retention.ms=60000
```

*Change a topic's retention on the fly*

### Groups

```
bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --all-groups
```

*See lag across every consumer group at once*

### Topics

```
bin/kafka-topics.sh --describe --bootstrap-server localhost:9092 --topic orders --exclude-internal
```

*Hide internal `__consumer_offsets`-style topics*

### Perf

```
bin/kafka-producer-perf-test.sh --topic orders --num-records 10000 --record-size 100 --throughput -1 --producer-props bootstrap.servers=localhost:9092
```

*Load-test throughput for a topic*

### Broker

```
bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092
```

*Confirm the broker is reachable and its API version*

# Stop Kafka, then ZooKeeper

Shut layers down in reverse order of how they started — Kafka depends on ZooKeeper, not the other way around.

```
● ● ● stop-kafka
```

```
# In the Kafka broker terminal  
Ctrl+C
```

```
$ jps  
(Kafka no longer listed)
```

```
● ● ● stop-zookeeper
```

```
# In the ZooKeeper terminal  
Ctrl+C
```

```
$ jps  
(QuorumPeerMain no longer listed)
```

## Step 12 — Stop Kafka

## Step 13 — Stop ZooKeeper

**Extra example — clean shutdown script for both:**

```
bin/kafka-server-stop.sh    # graceful stop without needing the original terminal  
bin/zookeeper-server-stop.sh
```

# One-page command cheat-sheet

## Cluster

- `start-dfs.sh / start-yarn.sh`
- `bin/zookeeper-server-start.sh config/zookeeper.properties`
- `bin/kafka-server-start.sh config/server.properties`

## Topics

- `--create --partitions N --replication-factor N`
- `--list`
- `--describe`
- `--alter --partitions N`
- `--delete`

## Producer

- `kafka-console-producer.sh --topic <t>`
- `--property parse.key=true`
- `--property key.separator=:`

## Consumer

- `kafka-console-consumer.sh --topic <t> --from-beginning`
- `--group <g>`
- `--max-messages N`
- `--partition N`

## Consumer Groups

- `kafka-consumer-groups.sh --list`
- `--describe --group <g>`
- `--reset-offsets --to-earliest --execute`

## Shutdown

- Ctrl+C in each terminal (reverse start order)
- `kafka-server-stop.sh`
- `zookeeper-server-stop.sh`

## YOUR TURN

# Start with Step 1 only

● ● ● your-turn.sh

```
$ java -version  
$ hadoop version  
$ jps
```

Send me the output of `jps`, and I'll guide you through starting ZooKeeper → Kafka → creating a topic → producer → consumer → deleting the topic → shutting everything down.

